

tucanoLinguagem

Base da linguagem:

Tucano será uma linguagem orientada a objetos, então haverá classes, herança, filhos e pais, além de tipagem forte porém não necessária.

A linguagem requer uma função `inicio` a ser declarada no arquivo principal do projeto, que possuirá o mesmo nome do projeto, com o sufixo `.tc` assim então um projeto nomeado `teste` possuirá o arquivo `teste.tc` que obrigatoriamente possui `funcao inicio` dentro

A linguagem utilizará `{}` para abrir e fechar laços de código, e `()` para determinar argumentos de função, sendo assim a função início deverá ser escrita como

```
funcao inicio() {  
  
}
```

Sendo uma linguagem escrita em português, o compilador ignorará caracteres especiais como ç e acentos em palavras reservadas, então `funcao`, `funçao`, `função` e `funcão` são equivalentes ao compilador.

A determinação de tipo de função ocorrerá com o uso de um `:` depois do fim dos argumentos, assim, por exemplo, uma função adição que soma dois números seria:

```
funcao soma(x:int,y:int):int {  
    retorna x+y  
}
```

pelo exemplo acima percebe-se que o `:` pode ser usado nos argumentos também para especificar seu tipo.

Em Tucano o caractere fim de linha `\n` é usado para interpretar o fim de um comando, logo ; é desnecessário, mas é uma opção para quando se quer escrever vários comandos em uma só linha, então

```
int x  
caractere y
```

e

```
int x; caractere y
```

executarão exatamente o mesmo código.

Comentários:

Em Tucano, os comentários são escritos com `--` antes do comentário e continua até o fim da linha `\n`

```
var x = 5 --isso é um comentário
--isso também.
```

Tipos de variáveis:

int:

Uma variável int é armazenada em código como um inteiro de 64 bits com sinal. Inicializado automaticamente em 0.

caractere:

Uma variável caractere é armazenada em código como uma String, ou um array de caracteres, o que for mais viável. Inicializado automaticamente em "".

lógico:

Uma variável lógico é armazenada em código como um boolean e possui dois estados Verdadeiro ou Falso. Inicializado automaticamente em falso.

real:

Uma variável real é armazenada como uma double em código, podendo armazenar números reais. Inicializado automaticamente em 0.0

dicionario<>:

Uma variável dicionário requer dois outros tipos dentro dos <>, o primeiro para ser a chave e o segundo para ser o valor.

Inicializado automaticamente em []

Declaração de variáveis:

A declaração de variáveis deve ser feita antes do uso da mesma, especificando ou não seu tipo, e inicializando ou não seu valor.

A palavra chave para criação de variáveis sem tipo é `var`.

```
var x = 10 --Variável do tipo int
var y = 10.0 --Variável do tipo real
var z = "oi" --Variável do tipo caractere

caractere teste = 10 --Erro de sintaxe: variável teste do tipo caractere
não pode receber valor do tipo int
```

Para declarar uma lista, se digita [] depois do tipo, se desejar um array de tamanho fixo, é só digitar o tamanho dentro dos colchetes, se não, deixe-os vazios que uma lista dinâmica será criada:

```
caractere[] listadecompras = {"Alface","Tomate"}
```

Não se pode fazer `var[]` dada a natureza da lista.

Matrizes podem ser alcançadas fazendo um truque simples:

```
dicionario<int[2],caractere> matriz
```

Essa linha de código cria um dicionário que usa dois valores inteiros para a chave, como se fosse uma coordenada

Um jeito melhor de criar matrizes deverá ser discutido.

Operadores:

Os operadores de comparação são os seguintes:

```
== igualdade
>= maior que ou igual a
<= menor que ou igual a
> maior que
< menor que
~= não é igual a
```

Importante lembrar que diferente do Javascript, todas as comparações em Tucano são estritas ao tipo, então `10=="10"` retornará falso.

O operador ternário existe em Tucano, sendo utilizado da seguinte forma:

```
logico éDia = falso
caractere relógio = éDia ? "Tá de dia" : "Tá de noite"
```

Os operadores matemáticos são os seguintes:

```
+ soma
- subtração
* multiplicação
/ divisão
^ exponenciação
// divisão inteira
% módulo, resto
.. concatenação
```

Os operadores lógicos são:

```
e
ou
xou
```

Os operadores de matemática discreta são:

```
& bitwise-and
| bitwise-or
~ negate
^| bitwise-xor
```

O operador `=` é usado para assinalar um valor a uma variável.

Podemos combiná-lo com os operadores matemáticos para encurtar as linhas de código:

```
+= soma
-= subtração
*= multiplicação
/= divisão
^= exponenciação
```

```
//= divisão inteira
%= módulo, resto
```

É importante lembrar que não podemos fazer isso com os operadores de matemática discreta, reservando-os apenas a escrita mais longa:

```
int x = 10
x += 5
-- x agora vale 15
x /= 4
-- x agora vale 3
x = x & 2
-- 11 & 10 = 10
-- x agora vale 2
x &= 3 --Erro de sintaxe: '&' inesperado encontrado.
```

Condições:

Usando o comando `se` podemos conferir uma condição:

```
logico sim = verdadeiro
se (sim) {
  --faça algo se for verdadeiro
}
```

Podemos também usar o `senão` para conferir o oposto da condição:

```
logico nao = falso
se (nao) {
  --faça algo se for verdadeiro
} senao {
  --faça algo se for falso
}
```

O `senão` pode ser combinado com um `se`:

```
logico nao = falso
se (nao) {
  --faça algo se for verdadeiro
} senao se (nao == falso) {
```

```
--faça algo se for falso  
}
```

Usando o comando `escolha..caso` podemos conferir várias condições:

```
int x = 10  
escolha (x) {  
  caso 1:  
    --faça algo  
  caso 2:  
    --faça outra coisa  
  caso padrão:  
    --faça isso  
}
```

Laços de repetição:

Usando o comando `para` podemos criar um laço de repetição finito:

```
para (i=1,10) faça {  
  --ambos os parametros são inclusivos  
}  
  
caractere[] lista = {"Alface","Batata"}  
para (vegetal de lista) faça {  
  --variavel vegetal vai iterar por todos itens da variavel lista  
}  
  
para (i=0,lista.tamanho()-1) faça {  
  var vegetal = lista[i]  
  --esse código é virtualmente idêntico ao mostrado acima  
}  
  
para (i=10,1) passo -1 faça {  
  --passo é usado para determinar o quanto o para deve avançar por  
  iteração  
  --quando o valor inicial é maior que o final e o passo não for  
  explicito, o padrão é -1  
}
```

```
para (i=0,20) passo 2 faca {  
    --esse para irá iterar por todos os números pares até o vinte  
}
```

Podemos também usar o laço `enquanto` de tal maneira:

```
int x = 1  
enquanto (x<10) {  
    --isso se repetirá enquanto x for menor que 10  
    --para isso ser útil, não se esqueça de incrementar o x no fim:  
    x += 1  
}
```

Podemos usar o `repita..até` também:

```
int x = 1  
repita até (x >= 10) {  
    --isso se repetirá até que x seja maior ou igual a 10.  
    --para isso ser útil, não se esqueça de incrementar o x no fim:  
    x += 1  
}
```

Podemos usar o `escapar` para sair de um laço de repetição:

```
int x = 1  
enquanto (verdade) {  
    --Este é um laço de repetição infinito.  
    x += 1  
    se (x >= 10) {  
        escapar --isso vai sair do laço se x for maior ou igual a 10.  
    }  
}
```

Entrada e Saída:

Embora não parte da linguagem em si, a Entrada e Saída vai ser gerenciada pelo pacote `util` nos objetos `Entrada` e `Saida`.

Você terá que criar esses objetos no código.

Porém esse pacote será importado automaticamente para você.

Exemplo de entrada:

```
Entrada e = novo Entrada()
funcao inicio() {
    caractere texto = e.lerlinha()
    --Esse código irá ler a primeira linha de texto que o usuário digitar no
    console
}
```

Exemplo de saída:

```
Saida s = novo Saida()
funcao inicio() {
    s.escreverlinha("Olá mundo!")
}
```

Visibilidade:

Por padrão, tudo na classe principal é público e tudo em outras classes é privado, mas você pode especificar usando `publico`, `privado` e `protegido`:

- Público: Todos os objetos podem acessar o atributo/método.
- Protegido: Somente o próprio objeto ou objetos filhos podem ter acesso.
- Privado: Somente o próprio objeto pode ter acesso.

Classes:

Para declarar uma classe você deve especificar seu nome e construtor, usando a palavra `classe` você pode criar a classe, com seus atributos e métodos, dos quais um método obrigatório será o `construtor` que é usado quando você instancia essa classe, o construtor deverá sempre retornar a classe que ele está:

```
classe Fruta {

    privado caractere nome

    publica funcao construtor(n:caractere):Fruta {
        isso.nome = n
        retorne isso
    }
}
```



```
}
```

Dentro da classe você pode usar o `isso` para se referir a instancia de objeto que está executando. Suponhamos a seguinte classe:

```
classe Fruta {  
  
    privado caractere nome  
  
    publico funcao construtor(n:caractere):Fruta {  
        isso.nome = n  
        retorne isso  
    }  
    publico funcao falarNome(s:Saida) {  
        s.escreverlinha(isso.nome) --escreva na tela o nome dessa fruta  
    }  
}  
  
funcao inicio() {  
    Saida s = novo Saida() --criar a saida  
  
    Fruta uva = novo Fruta("uva") --criar um objeto fruta com nome uva  
    Fruta melao = novo Fruta("melao") --criar um objeto fruta com nome melao  
  
    melao.falarNome(s) --o melao vai escrever "melao" na tela pois é essa  
    instancia que está executando.  
}
```

Gráficos:

Embora não seja parte da linguagem em si, faz parte da biblioteca padrão da linguagem, e fornece uma classe `Grafico` que pode ser instanciada:

```
funcao inicio() {  
    Grafico g = novo Grafico() --instancia uma janela de aplicativo padrão  
    800x600  
}
```

Com essa classe você pode fazer muita coisa, incluindo desenhar na tela:

```
funcao inicio() {  
    Grafico g = novo Grafico() --instancia uma janela de aplicativo padrão  
    800x600  
    g.mudarcor(0.5,0.5,1) --muda a cor para azul  
    g.retangulo("cheio",0,0,100,150) --desenha um retangulo nas coordenadas  
    0,0 de tamanho 100x150  
}
```